

Day 5

Module 8: Best Practices & Essentials

1. Project Structure & Clean Architecture (Layering)

Follow **separation of concerns** — never put business logic in controllers.

Recommended structure (DDD-lite / Clean Architecture inspired):

src/main/java/com/example/demo

— config	(security, beans, ...)
— controller	(REST endpoints – thin!)
— dto	(RequestDTO, ResponseDTO)
— entity	(JPA entities)
— exception	(custom exceptions + @RestControllerAdvice)
— mapper	(MapStruct or manual converters)
— repository	(Spring Data JPA)
— service	(business logic + @Service)
— util	(helpers, constants)

2. RESTful Design & HTTP Conventions

Rule	Good Example	Bad Example	Why it matters
Use nouns (resources)	<code>/api/v1/users</code> , <code>/api/v1/orders/123</code>	<code>/api/getUser</code> , <code>/doOrder</code>	True REST style
Plural for collections	<code>/users</code> , <code>/products</code>	<code>/user</code> , <code>/product-list</code>	Industry standard
Use hyphens in URLs	<code>/order-items</code>	<code>/orderItems</code> , <code>/order_items</code>	Better readability
Correct HTTP methods	GET, POST, PUT/PATCH, DELETE	POST everywhere	Semantic clarity
Proper status codes	201 Created + Location header	Always 200 OK	Client understands result
Versioning from beginning	<code>/api/v1/...</code> or header <code>Accept: ...</code>	No version → breaking changes	Future-proof

Most common mistake in 2025: Still using POST for everything and returning 200 for errors.

3. DTOs — Never Expose Entities

Very important rule:

- Create separate classes for **Request** and **Response**
- Never return JPA entities directly (lazy loading issues, security leaks, unnecessary fields)

Java

// Bad

```
@GetMapping("/{id}")  
public User getUser(@PathVariable Long id) { ... } // ← exposes entity
```

// Good

```
@GetMapping("/{id}")  
public UserResponseDto getUser(@PathVariable Long id) {  
    return userService.findById(id);  
}
```

Use **MapStruct** or **ModelMapper** for conversion — it saves a lot of boilerplate.

4. Validation (Always Validate Input!)

Use Jakarta Bean Validation + `@Valid`

Java



```
public record CreateUserRequest(  
    @NotBlank @Size(min=3, max=50) String username,  
    @Email String email,  
    @NotNull @Min(18) Integer age  
) {}
```

Java

```
@PostMapping  
public ResponseEntity<?> create(@Valid @RequestBody CreateUserRequest dto) { ... }
```

Add global exception handler for `MethodArgumentNotValidException`.

5. Exception Handling — Global & Consistent

Use `@RestControllerAdvice`

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(404, ex.getMessage(), LocalDateTime.now()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationErrors(MethodArgumentNotValidExcept
        // collect field errors...
    }
}
```

JSON

```
{  
  "status": 400,  
  "error": "Bad Request",  
  "message": "Validation failed",  
  "timestamp": "2026-03-03T10:46:00",  
  "path": "/api/v1/users"  
}
```

Group Activity

***M8GroupProject: Use the last API group activity and convert to SpringBoot and integrate to Postgres**